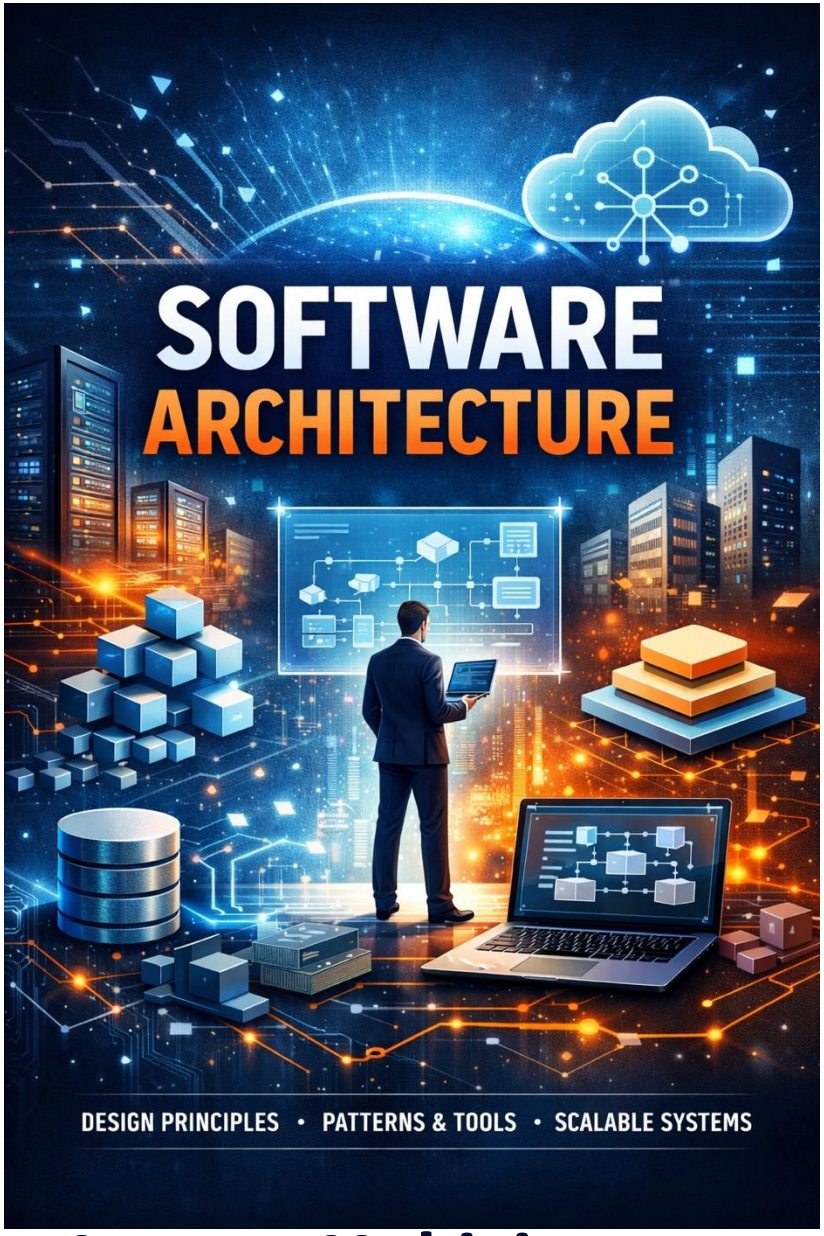
Software Architecture



Ignace Hakizimana

Overview

- Software systems today power governments, banks, healthcare institutions, startups, and global enterprises. Behind every reliable, scalable, and maintainable system lies a carefully designed architecture. This course introduces the principles, practices, tools, and strategic thinking required to design such systems effectively.

- Software architecture is not merely about writing code; it is about making critical structural decisions that determine how a system performs, evolves, and integrates within a broader ecosystem.

Overview

- This course is designed to equip you with both the theoretical foundations and the practical insight required to operate confidently at architectural levels whether in enterprise systems, cloud-native platforms, or distributed applications.

Overview

Why do some systems scale effortlessly while others fail under pressure?

- Because **software architecture** determines whether a system is built for growth or merely for immediate functionality.

- Architecture defines how components are structured, how workloads are distributed, how data is managed, and how failures are handled. A well-designed architecture anticipates increased traffic, evolving requirements, and integration complexity. It introduces clear boundaries, scalable patterns, resilience mechanisms, and performance safeguards from the outset. In contrast, systems that struggle under pressure often lack architectural foresight. They are tightly coupled, difficult to extend, and unprepared for sudden demand. What appears stable at a small scale becomes fragile when exposed to real-world load.

Course Objectives

By the end of this course, you will:

- ❖ Understand the meaning and significance of software architecture.
- ❖ Distinguish between enterprise, solution, and application architecture.
- ❖ Develop architectural thinking and decision-making skills.
- ❖ Apply design principles, architectural patterns, and best practices.
- ❖ Evaluate trade-offs in distributed and cloud-based systems.
- ❖ Communicate architectural decisions effectively to stakeholders.
- ❖ Analyze real-world systems through an architectural lens.

Why Software Architecture Matters

Poor architectural decisions lead to:

- Systems that cannot scale
- Security vulnerabilities
- Performance bottlenecks
- High maintenance costs
- Difficult integrations
- Frequent production failures

Strong architecture, on the other hand, enables:

- Scalability
- Reliability
- Maintainability
- Performance optimization
- Clear team collaboration
- Long-term sustainability

Course Structure

The course is organized into four structured lectures:

- **Lecture I: Course Introduction**

- **Lecture II: Introduction to Software Architecture:** Foundational concepts, architectural roles, and the three levels of architecture.

- **Lecture III: Essential Skills for Software Architects:** Core competencies including coding knowledge, documentation, decision-making, and stakeholder communication.

- **Lecture IV: Advanced Architecture and Tools:** Architectural patterns, distributed systems principles, SOLID, ACID, CAP Theorem, microservices, and modern tooling.

Conclusion

- Software architecture sits at the intersection of engineering, strategy, and design. It determines how systems behave, scale, and survive over time.

- This course will guide you through the structured development of architectural competence, from foundational principles to advanced distributed systems thinking.

Welcome to the journey of becoming a structured, strategic, and technically sound software architect.

End of the first lecture